

1) Define Comparator. Mention its methods and illustrate TreeSet in reverse order

Comparator is an interface in Java (java.util package) used to **compare two objects and define custom sorting order**.

Methods of Comparator

1. `int compare(Object o1, Object o2)`

→ Compares two objects

→ Returns: Negative → $o1 < o2$

Zero → $o1 = o2$, Positive → $o1 > o2$

2. `boolean equals(Object obj)`

- Checks equality of comparator

// Using a custom comparator

```
import java.util.*;
```

```
class MyComp implements Comparator<String> {public int compare(String a, String b)
{String aStr, bStr;aStr = a;bStr = b;return
bStr.compareTo(aStr); // reverse order}}
```

```
class CompDemo {public static void
main(String args[]) {TreeSet<String> t1 = new
TreeSet<String>(new MyComp());
```

```
t1.add("C");t1.add("A");t1.add("B");
```

```
t1.add("E");t1.add("F");t1.add("D");
```

```
for(String element : t1)
```

```
System.out.print(element + " ");
```

```
System.out.println();}}
```

Output: F E D C B A

Q2. Discuss the various methods provided by the Array class in Java. Illustrate with suitable example program.

The **Array class (java.util.Arrays)** provides methods to **manipulate arrays** such as sorting, searching, comparing, and filling.

Methods of Arrays Class

☐ **sort()** :Sorts the array in ascending order

☐ **binarySearch()**:Searches element in sorted array ,=>Returns index of element

☐ **equals()** Compares two arrays →Returns true if equal

☐ **fill()** →Fills array with a specific value

☐ **toString()** →Converts array to string format

Program example

```
import java.util.*;
```

```
class ArraysDemo {
```

```
    public static void main(String args[])
```

```
    {int arr[] = {5, 2, 8, 1, 3};
```

```
    // sort()
```

```
    Arrays.sort(arr);
```

```
    System.out.println("Sorted array: " +
Arrays.toString(arr));
```

```
    // binarySearch()
```

```
    int index = Arrays.binarySearch(arr, 3);
```

```
    System.out.println("Element 3 found at
index: " + index);
```

```
    // equals()
```

```
    int arr2[] = {1, 2, 3, 5, 8};
```

```
    System.out.println("Arrays equal: " +
Arrays.equals(arr, arr2));
```

```
    // fill()
```

```
    Arrays.fill(arr2, 10);
```

```
    System.out.println("After fill: " +
Arrays.toString(arr2));}}
```

Output

Sorted array: [1, 2, 3, 5, 8]

Element 3 found at index: 2

Arrays equal: true

After fill: [10, 10, 10, 10, 10]

3) Discuss Collection Algorithms in Java. Illustrate with suitable example.

"Algorithms operate on collections and are defined as static methods within the Collections class."

Collection Algorithms

1. **sort()** →Sorts elements in a list

2. **reverse()** →Reverses the order of elements

3. **shuffle()** →Randomly rearranges elements

4. **min()** →Returns minimum element

5. **max()** →Returns maximum element

6. **binarySearch()** →Searches element in sorted list

7. **fill()** Replaces all elements with a value

```
import java.util.*;
```

```
class AlgoDemo {
```

```
    public static void main(String args[]) {
```

```
        ArrayList<Integer> list = new ArrayList<>();
```

```
        list.add(10);list.add(30);list.add(20);list.add(50
);list.add(40);// sort()
```

```
        Collections.sort(list);System.out.println("Sorte
d: " + list);
```

```
        // reverse()
```

```
        Collections.reverse(list);
```

```
        System.out.println("Reversed: " + list);
```

```
        // max() and min()
```

```
System.out.println("Max: " + Collections.max(list));
System.out.println("Min: " + Collections.min(list));
// shuffle()
Collections.shuffle(list);
System.out.println("Shuffled: " + list);}}
```

Output:-

Sorted: [10, 20, 30, 40, 50]

Reversed: [50, 40, 30, 20, 10]

Max: 50

Min: 10

Shuffled: [30, 10, 50, 20, 40]

4. Describe all the string comparison methods available in Java with examples.

The String class includes several methods that compare strings or substrings within strings

String Comparison Methods

1. equals()

- Compares two strings for equality
- Case-sensitive

Syntax: boolean equals(Object str)

Example: String s1 = "Hello";

String s2 = "Hello";

String s3 = "hello";

System.out.println(s1.equals(s2)); // true

System.out.println(s1.equals(s3)); // false

2. equalsIgnoreCase()

Compares strings ignoring case

Syntax: boolean equalsIgnoreCase(String str)

Example: String s1 = "HelloWorld";

String s2 = "helloworld";

System.out.println(s1.equalsIgnoreCase(s2));
// true

3. regionMatches()

Compares specific part (region) of two strings

Syntax: boolean regionMatches(int start, String str2, int start2, int len)

Example: String s1 = "HelloWorld";

String s2 = "World";

System.out.println(s1.regionMatches(5, s2, 0, 5)); // true

4. startsWith()

Checks if string starts with given prefix

Syntax: boolean startsWith(String str)

Example: String s = "HelloWorld";

System.out.println(s.startsWith("Hello")); // true

5. endsWith()

Checks if string ends with given suffix

Syntax: boolean endsWith(String str)

Example: String s = "HelloWorld";

System.out.println(s.endsWith("World")); // true

Q5. Demonstrate the usage of string modification methods in Java with an example each.

String modification methods are used to **create new modified strings** from existing strings.

String Modification Methods

1. substring():=>Extracts part of a string

Syntax: String substring(int start, int end)

Example: String s = "JavaProgramming";

System.out.println(s.substring(0, 4)); // Java

2. replace():Replaces characters or substrings

Syntax: String replace(char old, char new)

Example: String s = "Java";

System.out.println(s.replace('a', 'o')); // Java

3. concat():=>Joins two strings

Syntax: String concat(String str)

Example: String s1 = "Hello";

String s2 = "World";

System.out.println(s1.concat(s2)); // HelloWorld

4. trim():=>Removes leading and trailing spaces

Syntax: String trim()

Example: String s = " Java ";

System.out.println(s.trim()); // Java

5. toUpperCase():=>Converts string to uppercase

Example: String s = "java";

System.out.println(s.toUpperCase()); // JAVA

6. toLowerCase():Converts string to lowercase

Example: String s = "JAVA";

System.out.println(s.toLowerCase()); // java

Q6. Explain the usage of indexOf() and lastIndexOf() methods with one example each

The indexOf() and lastIndexOf() methods are used to **search characters or substrings** in a string.

1. indexOf():>Returns the first occurrence of a character or substring

Syntax int indexOf(char ch)
int indexOf(String str)

Example class IndexOfExample {public static
void main(String args[]) {String s = "Java
Programming";int index = s.indexOf('a');

```
System.out.println("First occurrence of 'a': " + index);}}
```

Output:→ First occurrence of 'a': 1

2. lastIndexOf()→ Returns the **last occurrence** of a character or substring

Syntax:→ int lastIndexOf(char ch)
int lastIndexOf(String str)

Example

```
class LastIndexOfExample {public static void  
main(String args[]) {String s = "Java  
Programming";int index = s.lastIndexOf('a');  
System.out.println("Last occurrence of 'a': " +  
index);}}
```

Output:Last occurrence of 'a': 3

7) Illustrate the use of StringBuffer methods: append(), insert(), reverse(), capacity() and delete() with proper examples.

StringBuffer is a class used to create **mutable (modifiable) strings**.

1. append()→ Adds data at the end

Syntax:sb.append(data);

Example:

```
StringBuffer sb = new StringBuffer("Java");  
sb.append("Programming");  
System.out.println(sb);
```

Output:Java Programming

2. insert()→ Inserts data at specified position

Syntax:sb.insert(index, data);

Example:

```
StringBuffer sb = new StringBuffer("Java");  
sb.insert(4, "Language");  
System.out.println(sb);
```

Output:→ Java Language

3. reverse()→ Reverses the string

Syntax:sb.reverse();

Example:

```
StringBuffer sb = new StringBuffer("Java");  
sb.reverse();System.out.println(sb);
```

Output:aval

4. capacity()Returns current capacity of buffer

Syntax:sb.capacity();

Example:

```
StringBuffer sb = new StringBuffer();  
System.out.println(sb.capacity());
```

Output:16

5. delete()Deletes characters from given range

Syntax:sb.delete(start, end);

Example:

```
StringBuffer sb = new StringBuffer("Java  
Programming");
```

```
sb.delete(4,16);System.out.println(sb);
```

Output:Java

Q8. What are legacy classes? Explain any two legacy classes of Java's collection framework with suitable programs.

Legacy classes are the **old classes of Java (before Java 1.2)** that were later included in the **Collection Framework**.

Examples of Legacy Classes→ Vector, Stack, Hashtable, Dictionary (abstract)

1. Vector Class

- Vector is a **dynamic array**
- It can grow or shrink in size
- It is **synchronized**

```
import java.util.*;
```

```
class VectorDemo {  
    public static void main(String args[]) {  
        Vector<Integer> v = new Vector<>();  
        v.add(10); v.add(20);v.add(30);  
        System.out.println("Vector elements: " +  
v);v.remove(1); // remove element at index 1  
        System.out.println("After removal: " + v);}}
```

Output:→ Vector elements: [10, 20, 30]
After removal: [10, 30]

2. Stack Class

Stack follows **LIFO (Last In First Out)**

it extends **Vector class**

```
import java.util.*;
```

```
class StackDemo {  
    public static void main(String args[]) {  
        Stack<Integer> s = new Stack<>();  
        s.push(10);s.push(20);s.push(30);  
        System.out.println("Stack: " + s);  
        System.out.println("Popped element: " +  
s.pop());System.out.println("Top element: " +  
s.peek());}}
```

Output:→ Stack:[10,20,30]

Popped element: 30

Top element: 20

3. Hashtable

stores key-value pairs (no null key)

```
Hashtable<Integer, String> ht = new  
Hashtable<>();
```

```
ht.put(1,"A");ht.put(2,"B");System.out.println(  
ht);
```

9) What is Collection Framework? Explain methods of Collection, List, NavigableSet, Queue

Java Collection Framework is a set of **classes and interfaces** used to store and manipulate groups of objects.

1. Collection Interface

Important Methods

- add(E e) → adds element
- remove(Object o) → removes element
- size() → returns size
- isEmpty() → checks empty
- contains(Object o) → checks element
- clear() → removes all elements

2. List Interface

👉 Ordered collection (allows duplicates)

Methods

- add(int index, E element)
- get(int index)
- set(int index, E element)
- remove(int index)

3. NavigableSet Interface

👉 Sorted set with navigation methods

Methods

- first() → first element
- last() → last element
- higher(E e) → next greater
- lower(E e) → next smaller
- pollFirst() → removes first

4. Queue Interface

👉 Follows FIFO order

Methods

- add(E e) / offer(E e)
- remove() / poll()
- element() / peek()

10) Difference between String, StringBuffer, StringBuilder

Feature String StringBuffer StringBuilder

Mutability Immutable Mutable Mutable

Performance Slow Medium Fast

Thread			Not
Safety	Safe	Synchronized	synchronized

Memory More Less Less

```
String s = "Java";
s.concat(" World"); // new object
StringBuffer sb = new StringBuffer("Java");
sb.append(" World");
StringBuilder sb2 = new StringBuilder("Java");
sb2.append(" World");
```

10) Discuss various overloaded constructors of the String class with suitable code examples. Explain the behavior of each.

The String class in Java provides **multiple constructors** to create string objects in different ways.

Types of String Constructors

1. Empty Constructor

Syntax → String s = new String();

Explanation

- Creates an **empty string**
- Contains no characters

2. String from Character Array

Syntax String(char chars[])

Example → char ch[] = {'J','a','v','a'};
String s = new String(ch);
System.out.println(s); **Output** → Java

3. Subarray of Character Array

Syntax String(char chars[], int startIndex, int numChars)

Example char ch[] = {'a','b','c','d','e','f'};
String s = new String(ch, 2, 3);
System.out.println(s); **Output** → cde

4. Copy Constructor (from another String)

Syntax → String(String strObj)

Example → String s1 = "Java";
String s2 = new String(s1);
System.out.println(s1); System.out.println(s2);
Output → Java.Java

5. String from Byte Array

Syntax → String(byte ascii[], int start, int length)

Example → byte b[] = {65, 66, 67, 68, 69};
String s1 = new String(b);
String s2 = new String(b, 2, 3);
System.out.println(s1); System.out.println(s2);

Output

ABCDE
CDE